

21 — Await blocks: the promise's whole life, in markup

ASYNC TEMPLATE PROMISES

Q. The article fetch is still in flight — where do the spinner and the error message go, without a loading boolean and a try/catch in script?

ANSWER. Inside the `{#await}` block. It owns the promise's entire life in the template: the pending branch renders while the fetch is in flight, then exactly one of `{:then}` (the story landed) or `{:catch}` (it failed) replaces it. No booleans, no script glue, no manual state machine.

WHY IT WORKS. The instinct to hand-roll comes from plain JavaScript, where promise states are handled in `.then` and `.catch` callbacks, so the UI for them must be driven from script too. In Svelte the template can await directly, so the three states of the promise become three named places in the markup — the spinner has a home and the error has a home, and Svelte swaps between them for you.

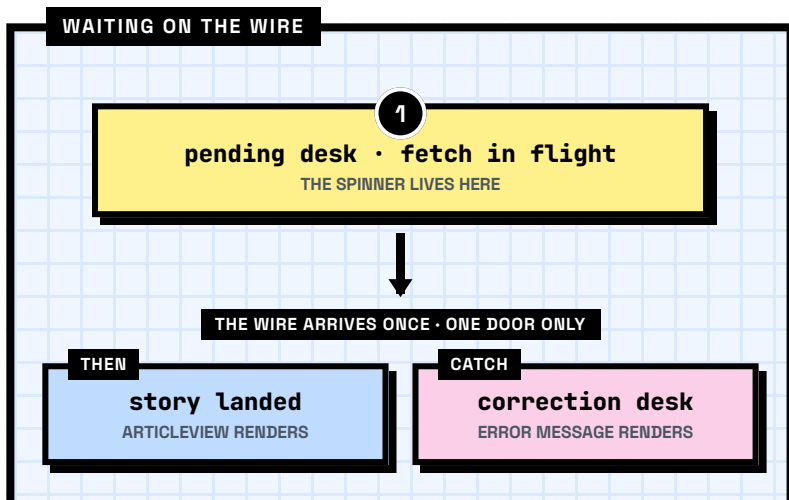
ArticlePage.svelte

```
<script>
  import { fetchArticle } from './api.js';
  let { id } = $props();
</script>

{#await fetchArticle(id)}
  <p>Loading the story...</p>                                <!-- pending:
in flight -->
{:then article}
  <ArticleView {article} />                                <!-- resolved
-->
{:catch err}
  <p>Could not load: {err.message}</p> <!-- rejected
-->
{/await}
```

DIAGRAM CONCEPT. Waiting on the Wire: the newsroom waits for the story at the pending desk; when the wire arrives it lands exactly once — on the editor's desk with

the story, or at the correction desk with the error.



- ① **Summary.** One block, three homes: pending holds the spinner, then holds the story, catch holds the error. The promise resolves once, Svelte swaps the branch, and your script stays free of loading booleans.